МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Комп’ютерний практикум № 2

з курсу «Основи розробки програмного забезпечення на платформі Microsoft.NET»

на тему: «LINQ to XML»

Викладач:
Ліщук К.І.

Виконав:
студент групи ІП-44
Комін Іван

Київ - 2026

Комп‘ютерний практикум № 2. LINQ to XML

Мета: - ознайомитися з обробкою XML документів з використанням технології LINQ to XML

Постановка задачі комп‘ютерного практикуму № 2

При виконанні комп‘ютерного практикуму необхідно виконати наступні дії:

1) Розробити структуру XML для зберігання даних згідно варіантів, наведених нижче.

2) Створити XML-файл з використанням XmlWriter. Дані необхідно вводити з консолі, зберегти.

3) Продемонструвати завантаження створеного в п.2 файлу (або заздалегідь створеного) з використанням:

a. XmlDocument;

b. XmlSerializer.

4) LINQ to XML:

a. Вивести зміст файлу, створеного в п.2 або заздалегідь створеного для демонстрації

b. Розробити як мінімум 6 різних запитів, використовуючи різні дії над отриманими даними. Запити не повинні повторюватись.

c. Розробити запити згідно варіанту згідно їх словесного опису.

5) Створити програмне забезпечення, котре реалізує обробку даних з використання бібліотеки LINQ to XML.

6) Програмне забезпечення необхідно розробити у вигляді консольного застосування на мові C#.

7) Звіт повинен містити: опис архітектури проекту, словесний опис запитів, текст програмного коду, скріншоти результатів виконання

Варіант індивідуальних завдань:

Розробити структуру даних для зберігання інформації для музичної платформи. Загальна характеристика задачі наступна:

• Користувачі слухають різні пісні.

• Пісні групуються в альбоми.

• Альбоми випускаються різними виконавцями.

• Виконавці записують багато пісень.

Розробити наступні запити:

• Визначити виконавців, що випустили найбільшу кількість альбомів, які були прослухані більше ніж 1 мільйон разів.

• Знайти пісні, які входять до найпопулярніших альбомів і мають найбільше прослуховувань

Опис архітектури проєкту:

1. Кореневий елемент (Root)

Весь документ загорнутий у тег <MusicPlatform>. Це логічний контейнер, який об’єднує всі сутності системи в єдиний простір імен.

2. Гілка «Каталог виконавців» (Artists)

Ця частина має трирівневу вкладеність, що відображає реальну структуру музичної індустрії:

- <Artist>: Визначає конкретного виконавця. Основна інформація (ім'я) винесена в атрибут Name.
- <Album>: Вкладений у тег артиста. Містить атрибути:
    - Title — назва;
    - Year — рік видання;
    - ListenCount — агреговане значення, яке обчислюється як сума прослуховувань усіх пісень усередині цього альбому.
- <Song>: Кінцевий елемент (leaf node). Всі характеристики треку (Duration, Plays, Rating) записані як атрибути. Це робить файл компактним і полегшує доступ до числових даних під час парсингу.

3. Гілка «Користувачі» (Users)

Цей блок існує паралельно з артистами всередині кореня. Така структура дозволяє уникнути надмірної вкладеності та дублювання даних.

- <User>: Визначає профіль користувача через атрибут Username.
- <HistoryItem>: Вкладений елемент, який містить лише посилання на назву треку (TrackTitle). Це реалізує логічний зв'язок між базою користувачів та каталогом пісень.

Приклад форматування XML-файлу:

```
<MusicPlatform>
  <Artist Name="Nirvana">
    <Album Title="Nevermind" Year="1991" ListenCount="1550000">
      <Song Title="Smells Like Teen Spirit" Plays="600000" Rating="5.0" />
      <Song Title="Lithium" Plays="350000" Rating="4.7" />
    </Album>
  </Artist>
  <User Username="Meloman99">
    <HistoryItem TrackTitle="Smells Like Teen Spirit" />
  </User>
</MusicPlatform>
```

Текст програмного коду для роботи із XML:

```
public static void WriteXml(string fileName)

    {

        var settings = new XmlWriterSettings { Indent = true };


        using XmlWriter writer = XmlWriter.Create(fileName, settings);


        writer.WriteStartDocument();

        writer.WriteStartElement("MusicPlatform");


        //input artists

        while (true)

        {

            Console.WriteLine("Enter artist name or 'exit' to quit");

            string artist = Console.ReadLine();

            if (artist?.ToLower() == "exit" ||
string.IsNullOrEmpty(artist))

                break;


            writer.WriteStartElement("Artist");
```

```
            writer.WriteAttributeString("Name", artist);


            //input albums
            while (true)
            {
                Console.WriteLine("Enter album name or 'exit' to quit");
                string album = Console.ReadLine();
                if (album?.ToLower() == "exit" ||
string.IsNullOrEmpty(album))
                    break;


                Console.WriteLine("Enter the release year");
                string year = Console.ReadLine();


                //xmlwriter works as a stream, so we save the sings into a
temp list to later dynamically count the ListenCount for the album
                var albumSongs = new List<(string Title, int Plays, string
Rating)>();


                //input songs
                while (true)
                {
                    Console.WriteLine("Enter song name or 'exit' to quit");
                    string song = Console.ReadLine();
                    if (song?.ToLower() == "exit" ||
string.IsNullOrEmpty(song))
                        break;


                    Console.WriteLine("Enter the amount of streams");
                    int.TryParse(Console.ReadLine(), out int plays);


                    Console.WriteLine("Enter the song rating");
```

```
                    string rating = Console.ReadLine();


                    albumSongs.Add((song, plays, rating));
                }


                long totalListenCount = albumSongs.Sum(s => (long)s.Plays);


                writer.WriteStartElement("Album");
                writer.WriteAttributeString("Title", album);
                writer.WriteAttributeString("Year", year);
                writer.WriteAttributeString("ListenCount",
totalListenCount.ToString());


                foreach (var s in albumSongs)
                {
                    writer.WriteStartElement("Song");
                    writer.WriteAttributeString("Title", s.Title);
                    writer.WriteAttributeString("Plays",
s.Plays.ToString());
                    writer.WriteAttributeString("Rating", s.Rating);
                    writer.WriteEndElement();
                }


                writer.WriteEndElement();
            }
            writer.WriteEndElement();
        }


        // input users
        while (true)
        {
```

```
            Console.WriteLine("Enter username or 'exit' to quit");
            string username = Console.ReadLine();
            if (username?.ToLower() == "exit" ||
string.IsNullOrEmpty(username))
                break;

            writer.WriteStartElement("User");
            writer.WriteAttributeString("Username", username);

            //input user history
            while (true)
            {
                Console.WriteLine("Enter song title from history or 'exit'
to quit");
                string historyTrack = Console.ReadLine();
                if (historyTrack?.ToLower() == "exit" ||
string.IsNullOrEmpty(historyTrack))
                    break;

                writer.WriteStartElement("HistoryItem");
                writer.WriteAttributeString("TrackTitle", historyTrack);
                writer.WriteEndElement();
            }

            writer.WriteEndElement();
        }

        writer.WriteEndElement();
        writer.WriteEndDocument();
    }
```

```
    public static void LoadWithXmlDocument(string fileName)
    {
        var doc = new XmlDocument();
        doc.Load(fileName);
        XmlNode root = doc.DocumentElement;

        Console.WriteLine("Loaded via XmlDocument:");

        Console.WriteLine("Artists and Albums:");
        foreach (XmlNode artist in root.SelectNodes("Artist"))
        {
            string name = artist.Attributes["Name"]?.Value;
            Console.WriteLine($"Artist: {name}");

            foreach (XmlNode album in artist.SelectNodes("Album"))
            {
                string title = album.Attributes["Title"]?.Value;
                Console.WriteLine($"  Album: {title}");
            }
        }

        Console.WriteLine("\nUsers and History:");
        foreach (XmlNode user in root.SelectNodes("User"))
        {
            string username = user.Attributes["Username"]?.Value;
            Console.WriteLine($"User: {username}");

            foreach (XmlNode historyItem in
user.SelectNodes("HistoryItem"))
            {
                string track = historyItem.Attributes["TrackTitle"]?.Value;
```

```
                Console.WriteLine($"  - Listened to: {track}");
            }
        }
    }

    public static void LoadWithSerializer(string fileName)
    {
        var serializer = new
System.Xml.Serialization.XmlSerializer(typeof(MusicPlatformDto));
        using (FileStream fs = new FileStream(fileName, FileMode.Open))
        {
            var data = (MusicPlatformDto)serializer.Deserialize(fs);

            Console.WriteLine("Loaded via Serializer:");

            foreach (var artist in data.Artists)
            {
                Console.WriteLine($"Artist: {artist.Name} (Albums:
{artist.Albums.Count})");
            }

            foreach (var user in data.Users)
            {
                Console.WriteLine($"User: {user.Username} (History:
{user.History.Count} items)");
            }
        }
    }

    public static void DisplayData(string filename)
    {
```

```
        var xmlDoc = XDocument.Load(filename);


        // artists
        Console.WriteLine("Artists and Albums:");
        foreach (XElement artist in xmlDoc.Root.Elements("Artist"))
        {
            Console.WriteLine($"Artist: {artist.Attribute("Name")?.Value}");
            foreach (XElement album in artist.Elements("Album"))
            {
                Console.WriteLine(
                    $"  - Album: {album.Attribute("Title")?.Value} ({album.Attribute("Year")?.Value})"
                );
                foreach (XElement song in album.Elements("Song"))
                {
                    Console.WriteLine(
                        $"    > {song.Attribute("Title")?.Value} - Plays: {song.Attribute("Plays")?.Value}"
                    );
                }
            }
        }


        // users
        Console.WriteLine("\nUsers and History:");
        foreach (XElement user in xmlDoc.Root.Elements("User"))
        {
            Console.WriteLine($"User: {user.Attribute("Username")?.Value}");
            foreach (XElement historyItem in user.Elements("HistoryItem"))
            {
```

```
                Console.WriteLine($"  - Listened to:
{historyItem.Attribute("TrackTitle")?.Value}");

            }

        }

    }
```

LINQ запити

1. Цей LINQ-запит виконує пошук усіх вузлів <User> у документі та фільтрує їх за наявністю конкретної пісні в історії прослуховувань. Використовуючи метод Descendants, програма обходить дерево XML, після чого для кожного користувача перевіряє вкладені елементи <HistoryItem> на відповідність назві "Smells Like Teen Spirit". Завдяки оператору Any, запит повертає імена лише тих слухачів, у яких знайдено хоча б один збіг, що дозволяє ефективно сегментувати аудиторію за їхніми вподобаннями.

```
string targetSong = "Smells Like Teen Spirit";

        var listeners = doc.Descendants("User")

            .Where(u =>

                u.Elements("HistoryItem").Any(h =>
h.Attribute("TrackTitle")?.Value == targetSong)

            )

            .Select(u => u.Attribute("Username")?.Value);
```

```
1. Users who listened to 'Smells Like Teen Spirit': Meloman99
```

2. Цей LINQ-запит виконує вибірку назв пісень, тривалість яких перевищує 300 секунд (5 хвилин). Програма знаходить усі елементи <Song> незалежно від їхньої вкладеності, фільтрує їх за числовим значенням атрибута Duration і повертає лише текстові значення атрибута Title. Це дозволяє швидко сформувати список довгих треків, відсіюючи композиції, що не відповідають заданому часовому критерію.

```
var longSongs = doc.Descendants("Song")
            .Where(s => (int?)s.Attribute("Duration") > 300)
            .Select(s => s.Attribute("Title")?.Value);
```

```
2. Songs longer than 5 minutes: Smells Like Teen Spirit, Time, Money, Us and Them, Experimental Noise
```

3. Цей LINQ-запит виконує хронологічне сортування всіх наявних у базі альбомів за роком їхнього релізу. Метод Descendants знаходить усі елементи <Album> незалежно від їхнього розташування в дереві XML, після чого оператор OrderBy впорядковує їх за зростанням числових значень атрибута Year.

```
var albums = doc.Descendants("Album")
            .OrderBy(a => (int?)a.Attribute("Year"))
            .Select(a => $"{a.Attribute("Title")?.Value}
({a.Attribute("Year")?.Value})");
```

```
3. Albums sorted by release year: The Dark Side of the Moon (1973), Nevermind (1991), In Utero (1993),
```

4. Цей LINQ-запит виконує агрегацію даних для обчислення загальної кількості прослуховувань на всій музичній платформі. Метод Descendants знаходить усі елементи <Song> у документі, після чого оператор Sum підсумовує значення їхніх атрибутів Plays, приведених до числового типу.

```
var totalStreams = doc.Descendants("Song").Sum(s =>
(int?)s.Attribute("Plays"));
```

```
4. Total amount of streams: 10600650
```

5. Цей LINQ-запит виконує пошук першого в документі альбому, який містить хоча б одну пісню з максимальним рейтингом.

```
var firstAlbum = doc.Descendants("Album")
            .FirstOrDefault(a => a.Elements("Song").Any(s =>
(double?)s.Attribute("Rating") == 5));
```

```
5. First album with the song rating of 5: Nevermind
```

6. Цей LINQ-запит виконує статистичне групування пісень за їхньою жанровою приналежністю для аналізу наповненості медіатеки. Метод Descendants збирає всі елементи <Song> у документі, після чого оператор GroupBy розподіляє їх на категорії згідно зі значенням атрибута GenreId.

```
var songsByGenre = doc.Descendants("Song")
            .GroupBy(s => s.Attribute("GenreId")?.Value)
            .Select(g => new { GenreId = g.Key, Count = g.Count() });
```

```
6. Groups of songs by genre:
   ID 1: 7 songs
   ID 2: 7 songs
   ID 3: 4 songs
   ID 4: 3 songs
```

7. Цей LINQ-запит виконує аналітичну вибірку для створення рейтингу найуспішніших виконавців за кількістю «мільйонних» альбомів. Метод Descendants знаходить усіх артистів у документі, після чого для кожного з них формується анонімний об'єкт, де через функцію Count підраховуються лише ті вкладені альбоми, чий атрибут ListenCount перевищує один мільйон.

```
var artistsWithMillionAlbums = doc.Descendants("Artist")
            .Select(artist => new
            {
                Name = artist.Attribute("Name")?.Value,
                MillionCount = artist
                    .Elements("Album")
                    .Count(album =>
(long?)album.Attribute("ListenCount") > 1000000),
            })
            .Where(a => a.MillionCount > 0)
            .OrderByDescending(a => a.MillionCount);
```

```
Artists with albums > 1,000,000 streams:
- Linkin Park: 2 album(s)
- Nirvana: 1 album(s)
- Pink Floyd: 1 album(s)
```

8. Цей комплексний LINQ-запит реалізує дворівневу аналітику для виявлення «хітів серед лідерів» платформи. Спочатку програма відбирає три найкращі альбоми за сумарним показником прослуховувань, після чого за допомогою методу SelectMany трансформує їхню ієрархічну структуру у плоский список усіх вкладених пісень. Отриманий перелік додатково сортується за індивідуальною популярністю треків, формуючи фінальний анонімний об'єкт із назвою пісні, кількістю її відтворень та назвою батьківського альбому для повного контексту результатів.

```
var query = doc.Descendants("Album")
```

```
            .OrderByDescending(a =>
(long?)a.Attribute("ListenCount"))
            .Take(3)
            .SelectMany(a => a.Elements("Song"))
            .OrderByDescending(s => (long?)s.Attribute("Plays"))
            .Select(s => new
            {
                Title = s.Attribute("Title")?.Value,
                Plays = s.Attribute("Plays")?.Value,
                AlbumTitle = s.Parent?.Attribute("Title")?.Value,
            });
```

```
Top songs from most popular albums:
- In the End (Album: Hybrid Theory) - 1200000 plays
- Numb (Album: Meteora) - 950000 plays
- Time (Album: The Dark Side of the Moon) - 900000 plays
- Money (Album: The Dark Side of the Moon) - 850000 plays
- Crawling (Album: Hybrid Theory) - 800000 plays
- Us and Them (Album: The Dark Side of the Moon) - 700000 plays
- Faint (Album: Meteora) - 700000 plays
- Somewhere I Belong (Album: Meteora) - 600000 plays
- Brain Damage (Album: The Dark Side of the Moon) - 500000 plays
- One Step Closer (Album: Hybrid Theory) - 500000 plays
- Papercut (Album: Hybrid Theory) - 400000 plays
```

Висновок: Реалізація музичної платформи засобами XML дозволила побудувати гнучку ієрархічну модель даних, де поєднання атрибутів та вкладених елементів забезпечило структуроване зберігання інформації про виконавців, альбоми та користувачів. Використання різнорівневих інструментів обробки — від потокового запису через XmlWriter до декларативних запитів LINQ to XML — продемонструвало високу ефективність у виконанні складної аналітики, зокрема автоматизованого підрахунку прослуховувань та фільтрації контенту за заданими критеріями. В результаті було створено надійну систему, яка гарантує цілісність даних завдяки типізації об'єктів та дозволяє проводити швидкий пошук і обробку медіатеки незалежно від обсягу вкладених елементів.